

```

<!--
{
  "availability" : [
    "iOS: 8.0.0 -",
    "iPadOS: 8.0.0 -",
    "macCatalyst: 13.0.0 -",
    "macOS: 10.10.0 -",
    "tvOS: 9.0.0 -",
    "visionOS: 1.0.0 -",
    "watchOS: 2.0.0 -"
  ],
  "documentType" : "symbol",
  "framework" : "Swift",
  "identifier" : "/documentation/Swift/Equatable",
  "metadataVersion" : "0.1.0",
  "role" : "Protocol",
  "symbol" : {
    "kind" : "Protocol",
    "modules" : [
      "Swift"
    ]
  },
  "preciseIdentifier" : "s:SQ"
},
{
  "title" : "Equatable"
}
-->

```

Equatable

A type that can be compared for value equality.

```

...

```

```

protocol Equatable : ~Copyable, ~Escapable

```

Overview

Types that conform to the `Equatable` protocol can be compared for equality using the equal-to operator (`==`) or inequality using the not-equal-to operator (`!=`). Most basic types in the Swift standard library conform to `Equatable`.

Some sequence and collection operations can be used more simply when the elements conform to `Equatable`. For example, to check whether an array contains a particular value, you can pass the value itself to the `contains(_)` method when the array's element conforms to `Equatable` instead of providing a closure that determines equivalence. The following example shows how the `contains(_)` method can be used with an array of strings.

```

...

```

```

let students = ["Kofi", "Abena", "Efua", "Kweku", "Akosua"]

```

```

let nameToCheck = "Kofi"
if students.contains(nameToCheck) {
    print("\(nameToCheck) is signed up!")
} else {
    print("No record of \(nameToCheck).")
}
// Prints "Kofi is signed up!"

```

Conforming to the Equatable Protocol

Adding ``Equatable`` conformance to your custom types means that you can use more convenient APIs when searching for particular instances in a collection. ``Equatable`` is also the base protocol for the ``Hashable`` and ``Comparable`` protocols, which allow more uses of your custom type, such as constructing sets or sorting the elements of a collection.

You can rely on automatic synthesis of the ``Equatable`` protocol's requirements for a custom type when you declare ``Equatable`` conformance in the type's original declaration and your type meets these criteria:

- For a ``struct``, all its stored properties must conform to ``Equatable``.
- For an ``enum``, all its associated values must conform to ``Equatable``. (An ``enum`` without associated values has ``Equatable`` conformance even without the declaration.)

To customize your type's ``Equatable`` conformance, to adopt ``Equatable`` in a type that doesn't meet the criteria listed above, or to extend an existing type to conform to ``Equatable``, implement the equal-to operator (``==``) as a static method of your type. The standard library provides an implementation for the not-equal-to operator (``!=``) for any ``Equatable`` type, which calls the custom ``==`` function and negates its result.

As an example, consider a ``StreetAddress`` class that holds the parts of a street address: a house or building number, the street name, and an optional unit number. Here's the initial declaration of the ``StreetAddress`` type:

```
...  
class StreetAddress {  
    let number: String  
    let street: String  
    let unit: String?  
  
    init(_ number: String, _ street: String, unit: String? = nil) {  
        self.number = number  
        self.street = street  
        self.unit = unit  
    }  
}
```

Now suppose you have an array of addresses that you need to check for a particular address. To use the ``contains(_:)`` method without including a closure in each call, extend the ``StreetAddress`` type to conform to ``Equatable``.

```
...  
extension StreetAddress: Equatable {  
    static func == (lhs: StreetAddress, rhs: StreetAddress) -> Bool {  
        return  
            lhs.number == rhs.number &&  
            lhs.street == rhs.street &&  
            lhs.unit == rhs.unit  
    }  
}
```

The ``StreetAddress`` type now conforms to ``Equatable``. You can use ``==`` to check for equality between any two instances or call the ``Equatable``-constrained ``contains(_:)`` method.

```

...
let addresses = [StreetAddress("1490", "Grove Street"),
                  StreetAddress("2119", "Maple Avenue"),
                  StreetAddress("1400", "16th Street")]
let home = StreetAddress("1400", "16th Street")

print(addresses[0] == home)
// Prints "false"
print(addresses.contains(home))
// Prints "true"
...

```

Equality implies substitutability—any two instances that compare equally can be used interchangeably in any code that depends on their values. To maintain substitutability, the `==` operator should take into account all visible aspects of an `Equatable` type. Exposing nonvalue aspects of `Equatable` types other than class identity is discouraged, and any that *are* exposed should be explicitly pointed out in documentation.

Since equality between instances of `Equatable` types is an equivalence relation, any of your custom types that conform to `Equatable` must satisfy three conditions, for any values `a`, `b`, and `c`:

- `a == a` is always `true` (Reflexivity)
- `a == b` implies `b == a` (Symmetry)
- `a == b` and `b == c` implies `a == c` (Transitivity)

Moreover, inequality is the inverse of equality, so any custom implementation of the `!=` operator must guarantee that `a != b` implies `!(a == b)`. The default implementation of the `!=` operator function satisfies this requirement.

Equality is Separate From Identity

The identity of a class instance is not part of an instance's value. Consider a class called `IntegerRef` that wraps an integer value. Here's the definition for `IntegerRef` and the `==` function that makes it conform to `Equatable`:

```

...
class IntegerRef: Equatable {
    let value: Int
    init(_ value: Int) {
        self.value = value
    }

    static func == (lhs: IntegerRef, rhs: IntegerRef) -> Bool {
        return lhs.value == rhs.value
    }
}
...

```

The implementation of the `==` function returns the same value whether its two arguments are the same instance or are two different instances with the same integer stored in their `value` properties. For example:

```

...
let a = IntegerRef(100)
let b = IntegerRef(100)

print(a == a, a == b, separator: ", ")

```

```
// Prints "true, true"
```

Class instance identity, on the other hand, is compared using the triple-equals identical-to operator (`===`). For example:

```
let c = a
print(c === a, c === b, separator: ", ")
// Prints "true, false"
```

Topics

Equatable Requirements

```
[`==( _:_: )`](/documentation/Swift/Equatable/==( _:_: ))
```

Returns a Boolean value indicating whether two values are equal.

```
[`!=( _:_: )`](/documentation/Swift/Equatable/!=( _:_: ))
```

Returns a Boolean value indicating whether two values are not equal.

Tuple Comparison

Compare tuples of between two and six `Equatable` elements for equality or inequality.

```
[`==( _:_: )`](/documentation/Swift/==( _:_: )-958in)
```

Returns a Boolean value indicating whether the corresponding components of two tuples are equal.

```
[`==( _:_: )`](/documentation/Swift/==( _:_: )-2htbb)
```

Returns a Boolean value indicating whether the corresponding components of two tuples are equal.

```
[`==( _:_: )`](/documentation/Swift/==( _:_: )-h88g)
```

Returns a Boolean value indicating whether the corresponding components of two tuples are equal.

```
[`==( _:_: )`](/documentation/Swift/==( _:_: )-7lhq7)
```

Returns a Boolean value indicating whether the corresponding components of two tuples are equal.

```
[`==( _:_: )`](/documentation/Swift/==( _:_: )-1hbor)
```

Returns a Boolean value indicating whether the corresponding components of two tuples are equal.

```
[`==( _:_: )`](/documentation/Swift/==( _:_: )-1ud2a)
```

Returns a Boolean value indicating whether the corresponding components of two tuples are equal.

```
[`==( _:_: )`](/documentation/Swift/==( _:_: )-9kf9y)
```

Returns a Boolean value indicating whether two types are identical.

`[`!=(_:_:)`](/documentation/Swift/!=(_:_:)-18co7)`

Returns a Boolean value indicating whether any corresponding components of the two tuples are not equal.

`[`!=(_:_:)`](/documentation/Swift/!=(_:_:)-7er1l)`

Returns a Boolean value indicating whether any corresponding components of the two tuples are not equal.

`[`!=(_:_:)`](/documentation/Swift/!=(_:_:)-754t2)`

Returns a Boolean value indicating whether any corresponding components of the two tuples are not equal.

`[`!=(_:_:)`](/documentation/Swift/!=(_:_:)-7ao4l)`

Returns a Boolean value indicating whether any corresponding components of the two tuples are not equal.

`[`!=(_:_:)`](/documentation/Swift/!=(_:_:)-4fzl6)`

Returns a Boolean value indicating whether any corresponding components of the two tuples are not equal.

`[`!=(_:_:)`](/documentation/Swift/!=(_:_:)-3nrcc)`

Returns a Boolean value indicating whether any corresponding components of the two tuples are not equal.

`[`!=(_:_:)`](/documentation/Swift/!=(_:_:)-1mxms)`

Returns a Boolean value indicating whether two types are not identical.

Pattern Matching

`[`~=(_:_:)`](/documentation/Swift/~=(_:_:))`

Returns a Boolean value indicating whether two arguments match by value equality.

Relationships

Conforming Types

`[`Int16`](/documentation/Swift/Int16)`

`[`Comparator`](/documentation/Swift/String/Comparator)`

`[`Index`](/documentation/Swift/Set/Index)`

`[`UInt128`](/documentation/Swift/UInt128)`

`[`Unit`](/documentation/Swift/Duration/UnitsFormatStyle/Unit)`

`[`SIMD16`](/documentation/Swift/SIMD16)`

`[`CapitalizationType-swift.enum`](/documentation/Swift/String/IntentInputOptions/CapitalizationType-swift.enum)`

`[`Instant`](/documentation/Swift/ContinuousClock/Instant)`

[`CollectionDifference`](/documentation/Swift/CollectionDifference)

[`UnownedTaskExecutor`](/documentation/Swift/UnownedTaskExecutor)

[`Double`](/documentation/Swift/Double)

[`Scalar`](/documentation/Swift/Unicode/Scalar)

[`UnsafePointer`](/documentation/Swift/UnsafePointer)

[`AnyKeyPath`](/documentation/Swift/AnyKeyPath)

[`Set`](/documentation/Swift/Set)

[`SIMD32`](/documentation/Swift/SIMD32)

[`DiscontiguousSlice`](/documentation/Swift/DiscontiguousSlice)

[`RemoteCallTarget`](/documentation/Distributed/RemoteCallTarget)

[`Index`](/documentation/Swift/FlattenSequence/Index)

[`AnyHashable`](/documentation/Swift/AnyHashable)

[`Duration`](/documentation/Swift/Duration)

[`Index`](/documentation/Swift/EnumeratedSequence/Index)

[`SIMDMask`](/documentation/Swift/SIMDMask)

[`Options`](/documentation/Observation/ObservationTracking/Options)

[`UInt8`](/documentation/Swift/UInt8)

[`AtomicUpdateOrdering`](/documentation/Synchronization/AtomicUpdateOrdering)

[`FloatingPointRoundingRule`](/documentation/Swift/FloatingPointRoundingRule)

[`WordPair`](/documentation/Synchronization/WordPair)

[`NumericType`](/documentation/Swift/Unicode/NumericType)

[`ClosedRange`](/documentation/Swift/ClosedRange)

[`UnitsFormatStyle`](/documentation/Swift/Duration/UnitsFormatStyle)

[`Dictionary`](/documentation/Swift/Dictionary)

[`CanonicalCombiningClass`](/documentation/Swift/Unicode/CanonicalCombiningClass)

[`JobPriority`](/documentation/Swift/JobPriority)

[`Int32`](/documentation/Swift/Int32)

[`SIMD4`](/documentation/Swift/SIMD4)

[`WritableKeyPath`](/documentation/Swift/WritableKeyPath)

[`SIMD8`](/documentation/Swift/SIMD8)

[`Attributed-swift.struct`](/documentation/Swift/Duration/UnitsFormatStyle/Attributed-swift.struct)

[`RangeSet`](/documentation/Swift/RangeSet)

[`ReferenceWritableKeyPath`](/documentation/Swift/ReferenceWritableKeyPath)

[`Index`](/documentation/Swift/LazyPrefixWhileSequence/Index)

[`UInt32`](/documentation/Swift/UInt32)

[`Character`](/documentation/Swift/Character)

[`Float16`](/documentation/Swift/Float16)

[`UniqueArray`](/documentation/Swift/UniqueArray)

[`CodingUserInfoKey`](/documentation/Swift/CodingUserInfoKey)

[`Index`](/documentation/Swift/ReversedCollection/Index)

[`Optional`](/documentation/Swift/Optional)

[`AnyIndex`](/documentation/Swift/AnyIndex)

[`Kind-swift.struct`](/documentation/Swift/Unicode/UTF8/ValidationError/Kind-swift.struct)

[`RegexSemanticLevel`](/documentation/Swift/RegexSemanticLevel)

[`Task`](/documentation/Swift/Task)

[`LocalTestingActorID`](/documentation/Distributed/LocalTestingActorID)

[`UnitWidth-swift.struct`](/documentation/Swift/Duration/UnitsFormatStyle/UnitWidth-swift.struct)

[`UInt16`](/documentation/Swift/UInt16)

[`Bool`](/documentation/Swift/Bool)

[`UnsafeRawPointer`](/documentation/Swift/UnsafeRawPointer)

[`FractionalPartDisplayStrategy`](/documentation/Swift/Duration/UnitsFormatStyle/FractionalPartDisplayStrategy)

[`Int`](/documentation/Swift/Int)

[`Publisher-swift.struct`](/documentation/Swift/Result/Publisher-swift.struct)

[`UnsafeCurrentTask`](/documentation/Swift/UnsafeCurrentTask)

[`UTF32`](/documentation/Swift/Unicode/UTF32)

[`RegexRepetitionBehavior`](/documentation/Swift/RegexRepetitionBehavior)

[`AtomicLoadOrdering`](/documentation/Synchronization/AtomicLoadOrdering)

[`Keys-swift.struct`](/documentation/Swift/Dictionary/Keys-swift.struct)

[`KeyboardType-swift.enum`](/documentation/Swift/String/TextInputOptions/KeyboardType-swift.enum)

[`ManagedBufferPointer`](/documentation/Swift/ManagedBufferPointer)

[`KeyPath`](/documentation/Swift/KeyPath)

[`Ranges-swift.struct`](/documentation/Swift/RangeSet/Ranges-swift.struct)

[`FloatingPointSign`](/documentation/Swift/FloatingPointSign)

[`Termination`](/documentation/Swift/AsyncStream/Continuation/Termination)

[`OpaquePointer`](/documentation/Swift/OpaquePointer)

[`Attributed-swift.struct`](/documentation/Swift/Duration/TimeFormatStyle/Attributed-swift.struct)

[`ObservationRegistrar`](/documentation/Observation/ObservationRegistrar)

[`Float80`](/documentation/Swift/Float80)

[`EmptyCollection`](/documentation/Swift/EmptyCollection)

[`SIMD64`](/documentation/Swift/SIMD64)

[`LocalizationValue`](/documentation/Swift/String/LocalizationValue)

[`Index`](/documentation/Swift/ClosedRange/Index)

[`ObjectIdentifier`](/documentation/Swift/ObjectIdentifier)

[`TimeFormatStyle`](/documentation/Swift/Duration/TimeFormatStyle)

[`StandardComparator`](/documentation/Swift/String/StandardComparator)

[`UnsafeMutableRawPointer`](/documentation/Swift/UnsafeMutableRawPointer)

[`ZeroValueUnitsDisplayStrategy`](/documentation/Swift/Duration/UnitsFormatStyle/ZeroValueUnitsDisplayStrategy)

[`CollectionOfOne`](/documentation/Swift/CollectionOfOne)

[`Placeholder`](/documentation/Swift/String/LocalizationValue/Placeholder)

[`ValidationError`](/documentation/Swift/Unicode/UTF8/ValidationError)

[`Instant`](/documentation/Swift/SuspendingClock/Instant)

[`Int8`](/documentation/Swift/Int8)

[`Float`](/documentation/Swift/Float)

[`Index`](/documentation/Swift/DiscontiguousSlice/Index)

[`Continuation`](/documentation/Swift/AsyncThrowingStream/Continuation)

[`Int64`](/documentation/Swift/Int64)

[`ErrorCode-swift.enum`](/documentation/Distributed/ExecuteDistributedTargetError/ErrorCode-swift.enum)

[`UInt64`](/documentation/Swift/UInt64)

[`FloatingPointClassification`](/documentation/Swift/FloatingPointClassification)

[`DisplayStyle-swift.enum`](/documentation/Swift/Mirror/DisplayStyle-swift.enum)

[`Index`](/documentation/Swift/CollectionDifference/Index)

[`AtomicStoreOrdering`](/documentation/Synchronization/AtomicStoreOrdering)

[`Int128`](/documentation/Swift/Int128)

[`ArraySlice`](/documentation/Swift/ArraySlice)

[`UInt`](/documentation/Swift/UInt)

[`Range`](/documentation/Swift/Range)

[`String`](/documentation/Swift/String)

[`Change`](/documentation/Swift/CollectionDifference/Change)

[`SIMD2`](/documentation/Swift/SIMD2)

[`SIMD3`](/documentation/Swift/SIMD3)

[`PartialKeyPath`](/documentation/Swift/PartialKeyPath)

[`UnicodeDecodingResult`](/documentation/Swift/UnicodeDecodingResult)

[`TaskPriority`](/documentation/Swift/TaskPriority)

[`GeneralCategory`](/documentation/Swift/Unicode/GeneralCategory)

[`UnsafeMutablePointer`](/documentation/Swift/UnsafeMutablePointer)

[`Index`](/documentation/Swift/Dictionary/Index)

[`Encoding`](/documentation/Swift/String/Encoding)

[`Kind-swift.struct`](/documentation/Observation/ObservationTracking/Event/Kind-swift.struct)

[`AutoreleasingUnsafeMutablePointer`](/documentation/Swift/AutoreleasingUnsafeMutablePointer)

[`Result`](/documentation/Swift/Result)

[`Pattern-swift.struct`](/documentation/Swift/Duration/TimeFormatStyle/Pattern-swift.struct)

[`RegexWordBoundaryKind`](/documentation/Swift/RegexWordBoundaryKind)

[`Continuation`](/documentation/Swift/AsyncStream/Continuation)

[`ContiguousArray`](/documentation/Swift/ContiguousArray)

[`Never`](/documentation/Swift/Never)

[`Publisher-swift.struct`](/documentation/Swift/Optional/Publisher-swift.struct)

[`ByteOrder`](/documentation/Swift/ByteOrder)

[`Index`](/documentation/Swift/String/Index)

[`Array`](/documentation/Swift/Array)

[`Substring`](/documentation/Swift/Substring)

Inherited By

[`SignedNumeric`](/documentation/Swift/SignedNumeric)

[`InstantProtocol`](/documentation/Swift/InstantProtocol)

[`Hashable`](/documentation/Swift/Hashable)

[`Strideable`](/documentation/Swift/Strideable)

[`Numeric`](/documentation/Swift/Numeric)

[`DurationProtocol`](/documentation/Swift/DurationProtocol)

[`UnsignedInteger`](/documentation/Swift/UnsignedInteger)

[`Comparable`](/documentation/Swift/Comparable)

[`BinaryFloatingPoint`](/documentation/Swift/BinaryFloatingPoint)

[`FixedWidthInteger`](/documentation/Swift/FixedWidthInteger)

[`BinaryInteger`](/documentation/Swift/BinaryInteger)

[`SIMD`](/documentation/Swift/SIMD)

[`AdditiveArithmetic`](/documentation/Swift/AdditiveArithmetic)

[`SignedInteger`](/documentation/Swift/SignedInteger)

[`OptionSet`](/documentation/Swift/OptionSet)

[`DistributedActor`](/documentation/Distributed/DistributedActor)

[`SetAlgebra`](/documentation/Swift/SetAlgebra)

[`StringProtocol`](/documentation/Swift/StringProtocol)

[`FloatingPoint`](/documentation/Swift/FloatingPoint)

Copyright © 2026 Apple Inc. All rights reserved. | [Terms of Use](https://www.apple.com/legal/internet-services/terms/site.html) | [Privacy Policy](https://www.apple.com/privacy/privacy-policy)